



AI Theater Planner

Personalized Theater Recommendation & Planning Agent

MATH 690 - Graduate Seminar : Agentic AI Project

Students: Ayşegül Yavuz, Makbule Özge Özler

Instructor: Dr. Erdem Özcan

Due Date: December 2025



Agenda

1. Problem Statement
2. Solution Overview
3. Technical Architecture
4. Prompting Techniques Used
5. System Components
6. Demo Scenario
7. Project Management Timeline
8. Evaluation Metrics
9. Learning Outcomes
10. Future Work
11. References
12. Q&A

Problem Statement

Current Challenges

Theater enthusiasts face multiple pain points:

-  Hard to track which plays are showing when
-  Difficult to filter by location/distance
-  No personalized recommendations based on past preferences
-  Missing context (actor interviews, reviews)
-  Manual calendar coordination

Our Solution

An AI agent that:

- Syncs with your calendar
- Recommends plays based on your taste
- Filters by location and availability
- Suggests pre/post-show content (interviews, reviews)



Solution Overview

Core Features

1. Calendar Integration

- Automatically finds free time slots
- Avoids scheduling conflicts

2. Intelligent Filtering

- Location-based (max 30 min travel)
- Genre preferences (drama vs. comedy)
- Venue preferences (state theater vs. private)

3. Personalized Scoring

- Analyzes past ratings
- Predicts enjoyment likelihood

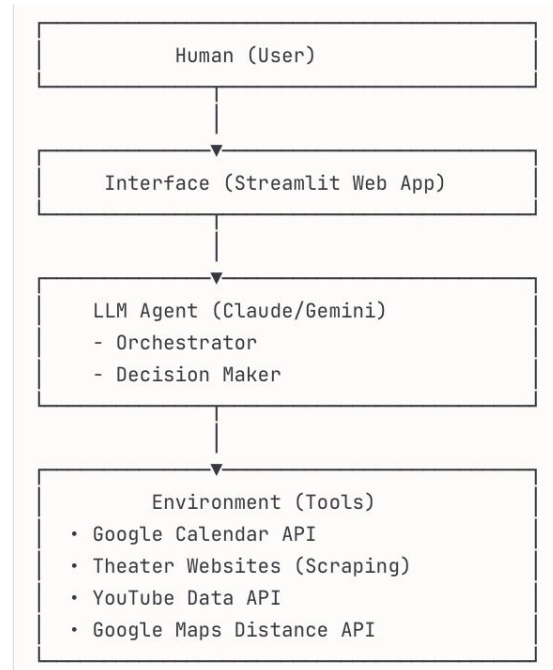
4. Content Discovery

- Actor interviews (YouTube)
- Reviews and articles
- Behind-the-scenes content



Technical Architecture

High-Level Design (From GSU - Agents 2 Lecture Slide 16)





Prompting Techniques Used

1. Zero-Shot (from Slide 6)

Use Case: Simple queries

Examples: "What plays are showing today in Istanbul?"

2. Few-Shot (from Slide 7)

Use Case: Preference prediction

Examples:

- "Hamlet" → Liked (Drama, Classic)

- "Comedy Night" → Disliked (Shallow humor)

Predict: Will user like "King Lear"?

3. Chain-of-Thought (from Slide 8)

Use Case: Multi-step reasoning

1. Analyze user's calendar 2. Find available time slots 3. Filter plays by location 4. Score based on past preferences 5. Rank and recommend



System Components

Tool Calling (Slide 12)

Possible Available Tools:

```
# Calendar Management
get_calendar_events(date_range)

# Theater Data
search_plays(city, date)
get_play_details(play_id)

# Location Services
calculate_distance(user_location, venue)

# Content Discovery
search_interviews(play_name, actor)
get_reviews(play_name)

# User Preferences
get_user_ratings(user_id)
save_user_rating(play_id, rating)
```

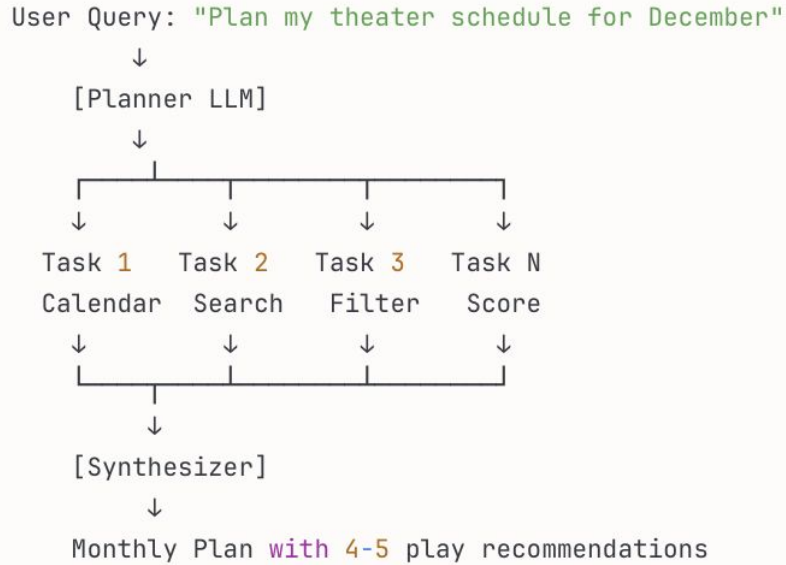


System Components



Agentic Architecture Pattern

Planner Pattern (Slide 23)



**Why Planner?

- Complex task requiring multiple steps
- Tasks can be executed **in** sequence
- Plan needs to be tracked across iterations



System Components



Reflection Loop (Slide 21)

Continuous Improvement

User: "Recommend a play"



[Generator]: "I suggest 'Hamlet'"



[Reflector]: "Check if user likes tragedy"



User Feedback: ★★★★★



[Update Model]: Increase weight for tragedy genre

**Benefits:

- Learns from user feedback
- Improves over time
- Self-correcting system



System Components



Technical Stack

Backend

- **Language:** Python 3.11+
- **LLM Integration:** LiteLLM (multi-provider)
- **Models:** Claude 3.5 Haiku, Gemini 2.0 Flash
- **APIs:** - Google Calendar API - YouTube Data API v3 - Google Maps Distance Matrix API



System Components



Technical Stack

Frontend

- **Framework:** Streamlit
- **Deployment:** Streamlit Cloud

Data Storage

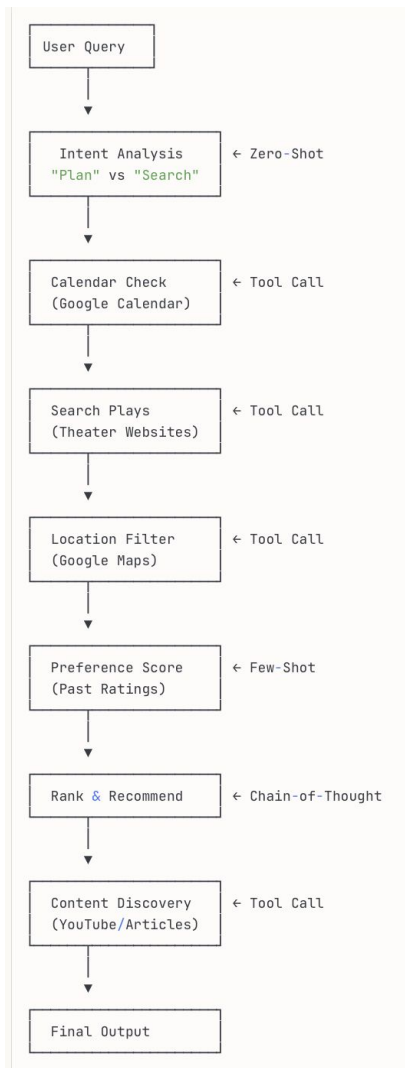
- **User Preferences:** SQLite
- **Cache:** Redis (for API responses)



System Components



System Flow Diagram



Project & Risk Management



Future Work

Phase 1 (Current Project)

-  Single city (Istanbul)
-  Manual calendar sync
-  Basic preferences

Phase 2

-  Multi-city support
-  International festivals
-  Travel cost estimation
-  Hotel/flight booking

Phase 3 (Advanced Features)

-  Group recommendations (friends)
-  Automatic ticket purchasing
-  Mobile app (React Native)
-  Multi-language support



Evaluation Metrics

Accuracy

- **Recommendation Hit Rate:** (% of recommended plays actually attended)
- **Target:** (i.e. 70% and above)

User Satisfaction

- **Average Rating:** (User feedback on recommendations)
- **Target:** (i.e. 4.2/5.0)

System Performance

- **Response Time:** (Time to generate plan)
- **Target:** (i.e. < 10 seconds)

Coverage

- **Play Database:** Number of plays tracked
- **Target:** (i.e. 50 plus active plays)

(Expected) Learning Outcomes

1. Agentic AI Patterns

- Planner architecture
- Tool calling integration
- Reflection loops

2. Prompting Techniques

- Zero-shot **for** quick queries
- Few-shot **for** preference learning
- Chain-of-thought **for complex** reasoning

3. Real-World Integration

- API integration (Google Calendar, Maps)
- Web scraping (theater sites)
- Data persistence (user preferences)

References

Course Materials:

- Lecture Slides: "Prompting LLMs" (Week 3)
- Lecture Slides: "Architecture"(Week 4)

Technical Resources:

- Anthropic Claude Documentation
- LiteLLM Documentation - Google Calendar API v3
- YouTube Data API v3

Inspiration:

- Erdem Özcan

 Thank You!

Contact

Ayşegül Yavuz

Email: mailtoayz@gmail.com GitHub: <https://github.com/ayz-90>

Makbule Özge Özler

Email: mak.ozgeozler@gmail.com GitHub: github.com/ozgeozler93



Will be Available At [theater-planner-demo.streamlit.app]



Possible Common Questions:

Q1: How do you handle multiple users?

A1: Each user has isolated preference profile in database

Q2: What if play info is incorrect?

A2: Reflection loop allows users to report errors, system self-corrects

Q3: Privacy concerns with calendar access?

A3: OAuth2, read-only access, user can revoke anytime

Q4: Cost of API calls?

A4: Cached responses, batched requests, ~\$5/month for 100 users